

User Manual for the package sampley – sample survey data

version 0.0.16

May 2026

Authors

The sampley package has been developed and is maintained by Jonathan Syme¹ with the assistance of Daniel Pendleton^{1, 2}, Nicholas Record¹, Benjamin Tupper¹, and Erin Meyer-Gutbrod³.

Any enquiries or queries should be addressed to Jonathan Syme at jsymect@gmail.com

¹Bigelow Laboratory for Ocean Sciences, East Boothbay, ME, USA

²NOAA Northeast Fisheries Science Center, Woods Hole, MA, USA

³School of the Earth, Ocean and Environment, University of South Carolina, Columbia, SC, 29208

Description

The sampley package serves to sample survey data (hence ‘sampley’; sample + survey). By ‘survey data’, we refer principally to systematic visual survey data, although the sampley package may also be applicable to other kinds of survey data. By ‘sample’, we mean process those data to produce distinct samples that can then be put into a model. For more information, please consult the General Notes section of this User Manual or the associated paper (<https://doi.org/10.1111/2041-210x.70320>)

Citation

To cite sampley, please cite the following paper:

Syme, J., Pendleton, D. E., Meyer-Gutbrod, E. L., Tupper, B., & Record, N. R. (2026). sampley: A Python package for sampling visual survey data. *Methods in Ecology and Evolution*. <https://doi.org/10.1111/2041-210x.70320>

You can also cite the package directly with:

Syme, J., Pendleton, D. E., Meyer-Gutbrod, E. L., Tupper, B., & Record, N. R. (2026). sampley: sample survey data (v0.0.16). <https://doi.org/10.5281/zenodo.19616964>

Contents

Setup	3
Python.....	3
Jupyter	3
sampley	3
General Notes	4
The grid, segment, and point approaches	4
Data	4
Stages.....	5
Classes and Objects.....	5
Attributes	5
Methods	6
Export	6
Stages	7
Paths.....	8
Stage 1 – importing data	9
DataPoints	10
Sections	14
Stage 2 – delimiting	18
Periods	19
Cells	21
Segments.....	24
Presences	27
PresenceZones	29
Absences	32
Stage 3 – sampling.....	36
Samples	37
Additional functions	46

Setup

To install sampley, the user will need to install Python and then, preferably, Jupyter (or another Python IDE of their choice) before installing sampley itself. To run sampley, a basic knowledge of Python is ideal, however exemplar Jupyter Notebooks are provided in the repository so that users with only limited knowledge of Python will still be able to run it.

Below we provide guidance and links to some of the official resources from Python and Jupyter on download and installation as well as some introductory tutorials. Additionally, an abundance of unofficial guides and tutorials, both free and paid, can be found online.

Python

Python is free to download from the official download page: <https://www.python.org/downloads/>. Note that sampley requires Python version ≥ 3.8 . The official Python Tutorial can be found here: <https://docs.python.org/3/tutorial/index.html>. Numerous unofficial tutorials are also available online.

Jupyter

Jupyter can be installed as per the instructions found here: <https://jupyter.org/install>. Both Jupyter Notebook and Jupyter Lab (which includes Notebooks) are suitable, although a beginner may find the simplicity of Jupyter Notebook easier to use. Documentation on both Jupyter Notebook and Jupyter Lab can be found here: <https://docs.jupyter.org/en/latest/>.

Alternative IDEs include, but are not limited to, Visual Studio Code (<https://code.visualstudio.com/>) and PyCharm (<https://www.jetbrains.com/pycharm/>).

sampley

The sampley package can be freely installed from PyPI by using one of the following options:

In command line: `pip install sampley`

In Jupyter: `%pip install sampley`

To import sampley (the equivalent of loading a package in R), use one of the following lines:

For general use: `from sampley import *`

For accessing the underlying functions: `from sampley.functions import *`

A link to the repository and exemplar notebooks, can be found on the sampley PyPI Project Page: <https://pypi.org/project/sampley/>.

To get information on any method, run `help(METHOD)` where METHOD is the method's name without parentheses. For example, to get information on the `Sections.from_file()` method, run: `help(Sections.from_file)`.

General Notes

The grid, segment, and point approaches

In some studies, surveys are conducted throughout a given area, either along transects or at random, with sampling units created afterwards by dividing the survey effort and detections spatially and temporally. In general, this division is achieved by one of three main approaches – referred to here as the grid approach, the segment approach, and the point approach – each with its own variations.

The grid approach consists of overlaying a grid, typically rectangular or hexagonal, onto the study area and allocating detections and, optionally, survey effort to the cells that they lie within. Additionally, data may be allocated to temporal periods. Each cell within a given period then serves as a sample.

The segment approach involves taking sections of continuous, uniform survey effort and cutting them into segments of standardised lengths which serve as samples.

The point approach consists of using the detections, or a subset thereof, as presences and then sampling absences from absence zones (i.e., areas that were surveyed but where the species was not detected).

Data

The sampley package has been developed to process data collected during visual surveys for species great and small (e.g., line-transect surveys) into a form that can be imported to a model (e.g., a species distribution model). Nevertheless, it may also be applicable to other kinds of survey data (e.g., non-systematic surveys or acoustic surveys), assuming they meet the following requirements.

At a base level, what is required is a survey track (i.e., the route taken by the observer(s) during a survey) and datapoints that were collected during that survey (e.g., sightings or in-situ recordings of environmental variables). As the data are collected during a survey, they should be ‘on-line’ (i.e., on the survey track), with the potential exception of sightings, which may be ‘off-line’ if their location is determined, for example, via distance sampling.

The sampley package is not meant for extracting or processing data from other sources (e.g., sea surface temperature data from a satellite database or depth measurements from a bathymetry layer), although it does provide the parameters necessary to facilitate these steps once sampling has been completed.

The sampley package does not provide custom functionality to filter or subset data, although standard Python functionality is available. Accordingly, it is recommended to only import those data that are to be processed and to quality check data (e.g., check for duplicate datapoints, check for datapoints missing data) and remove any unwanted datapoints (e.g., off-effort survey track, sightings with low certainty) beforehand.

Stages

The process followed when implementing the sampley package follows three stages:

Stage 1 – importing

Stage 2 – delimiting

Stage 3 – sampling

The manual is structured to follow these three stages with more information on each stage found in each corresponding section of the manual.

Classes and Objects

The sampley package contains 9 custom classes. Some or all of these classes may be used during processing, depending on the user's requirements. The classes are listed below according to the stage in which they are made:

Stage 1: DataPoints, Sections

Stage 2: Periods, Cells, Segments, Presences, PresenceZones, Absences

Stage 3: Samples

More detail on each class is given under their respective Stage subheadings.

In this manual, an instance of a class is referred to as an object (e.g., an instance of a DataPoints class is referred to as a DataPoints object). Objects are always written with an initial uppercase, while the values that they contain have the same name but are written with lowercase (e.g., 'Segments' refers to a Segments object, whereas 'segments' refers to the segments themselves).

Attributes

Each object has a *name* attribute and a *parameters* attribute which contain the name of the object and key parameters relating to it, respectively.

Most objects have a third attribute called the same as the class but in lower case (e.g., a DataPoints object has a *datapoints* attribute, a Cells object has a *cells* attribute) that contains the values within a dataframe (specifically, a pandas.DataFrame or a geopandas.GeoDataFrame). Within this dataframe, each row is a value of that type (e.g., a DataPoints object has a datapoints attribute which is a dataframe where each row is a datapoint, a Segments object has a segments attribute which is a dataframe where each row is a segment).

To access a given attribute, use the object's name, a full stop, and the attribute's name (e.g., to access the parameters attribute of an object called u_datapoints, use u_datapoints.parameters).

Further detail on attributes for each class is given later in the manual.

Methods

Each class has methods that can be divided into two categories: class methods and standard methods. Class methods are used to make an object of that class. For example, to make a `DataPoints` object, the user can use the *from_file* class method. Broadly speaking, the classes in Stage 1 have a *from_file* class method, those in Stage 2 have a *delimit* class method, while the `Samples` class has various class methods depending on the approach used.

Standard methods are used to apply a function to an object that has already been made. The standard methods vary between classes though most have a *plot* and a *save* method to plot and save the object, respectively.

Export

To facilitate the saving of objects, an export folder should be created where all exported objects will be saved. The name of each saved file will be derived automatically from the name of the object. The name of an object can be modified manually by editing the *name* attribute.

Stages

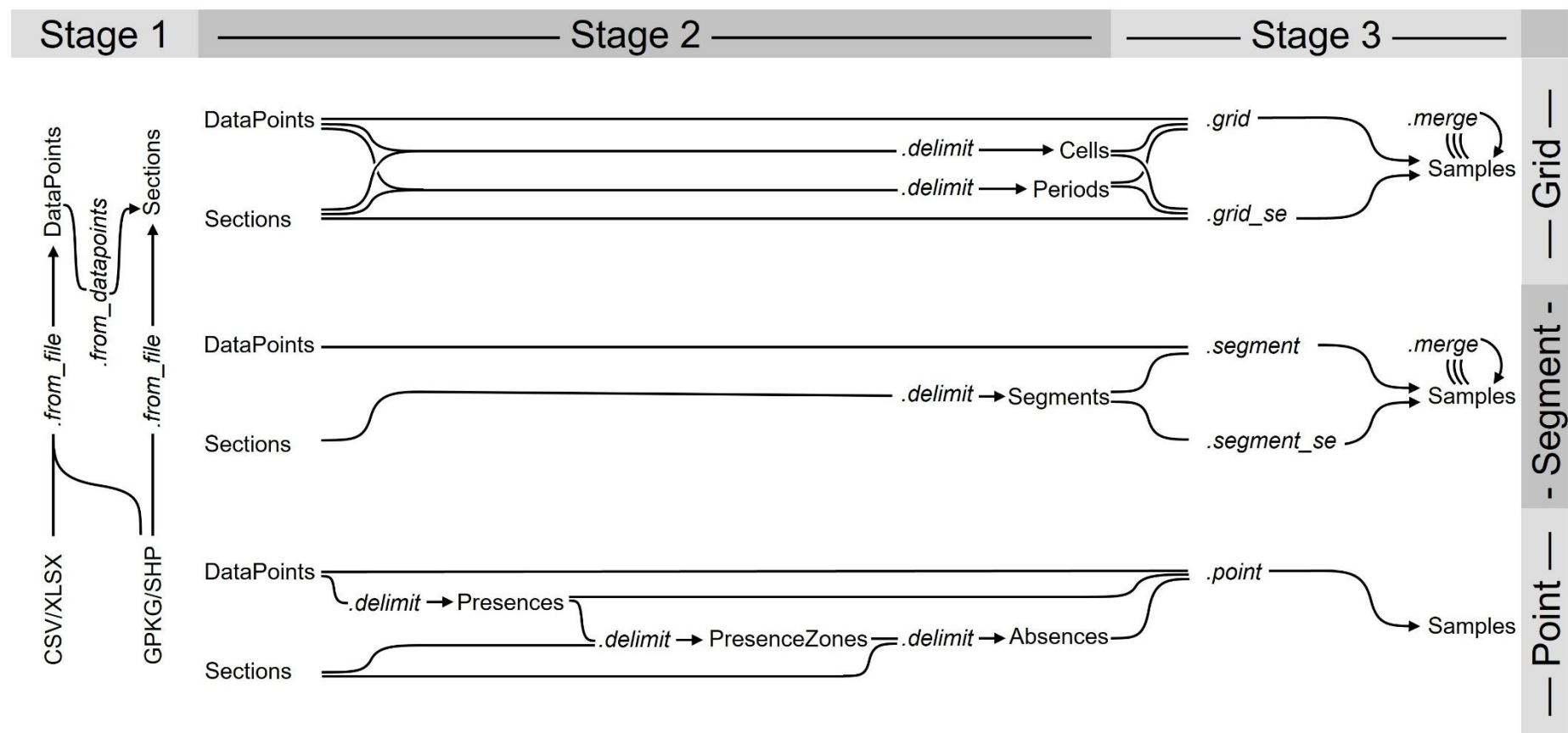
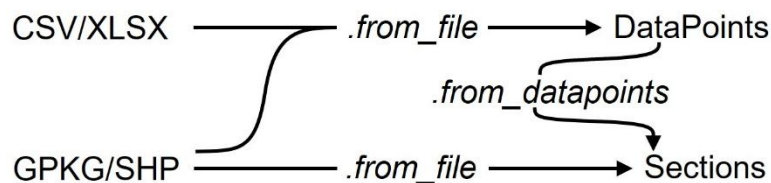


Fig 1. A graphical representation of the three Stages of data processing with `sampley`. Stage 1 is the same for all approaches, but Stages 2 and 3 follow different paths according to the choice of grid, segment, or point approach. Objects are written in normal font (e.g., `DataPoints`, `Sections`) while methods are written in italics (e.g., `.from_file`, `.from_datapoints`). The methods belong to the object that their arrow points to (e.g., `.from_datapoints` belongs to `Sections`). A detailed explanation of this figure, and similar figures found throughout, is provided on the next page.

Paths

Throughout the manual, smaller portions of Fig. 1 are presented, such as the example below:



These figures illustrate the possible paths that can be taken to conduct data processing with *sampley* and illustrate which objects and methods will be required. For example, the above figure shows, amongst other things, that a GeoPackage (GPKG) can be imported with the *from_file* method to make a Sections object.

For all these figures, please note the following:

Objects are written in normal font (e.g., DataPoints, Sections), class methods for making those objects and standard methods for modifying them are written in italics (e.g., *.from_file*, *.from_datapoints*). The methods belong to the object that their arrow points to (e.g., *.from_datapoints* belongs to Sections).

Lines indicate which methods each object is entered into or produced by. If two lines merge before a method (e.g., the lines from CSV/XLSX and GPKG/SHP merge before entering *.from_file*), then either of those objects can be entered. If the lines stay separate, then both must be entered. In addition to the entered objects, each method will require values for several parameters that are not included in the diagram but are detailed in the corresponding parts of the manual.

All objects have a *.open* and a *.save* method that are not illustrated in the paths while all except Periods objects have a *.plot* method to make a simple plot. These methods, along with some additional methods for certain objects, are detailed in the corresponding parts of the manual.

Stage 1 – importing data

To conduct processing, each imported file has to be wrangled into one of two classes of objects: Sections and/or DataPoints. In most scenarios, a Sections object and at least one DataPoints object will be necessary to conduct processing. Once any Sections and/or DataPoints objects have been made, they will form the basis from which all subsequent objects are derived. Moreover, any Sections and/or DataPoints objects will not change during processing. This allows for an unlimited number of objects to be derived from them (e.g., if the user wants to process the data via more than one approach).

Import filetypes

The imported files can be any of the following types: GeoPackage (GPKG), Shapefile (SHP), Comma-separated values (CSV), or Microsoft Excel (XLSX) – although use of GPKGs and CSVs is recommended. The importing process, as well as all subsequent processing, is the same for all filetypes.

Coordinate Reference System (CRS)

Use of a projected CRS, preferably one that preserves distance and uses metres, is recommended in all cases and essential for cases involving distance measurements. If the imported files are not in an appropriate CRS, they can be reprojected during importing. Please note that objects with different CRSs will be incompatible.

Timezone (TZ)

Imported files may have datetimes, dates, or no temporal data, although some options will not be possible if temporal data are absent. If imported files have datetimes or dates, timezones can be set and, additionally, datetimes can be converted to a different timezone during importing. Please note that objects with different timezones will be incompatible.

Paths

A DataPoints object can be made from a CSV, XLSX, GPKG, or SHP file by using the DataPoints.from_file class method (Fig 2). A Sections object can be made from a GPKG or SHP file by using the Sections.from_file class method (Fig 2). Alternatively, a Sections object can be made from a DataPoints object by using the Sections.from_datapoints class method (Fig 2).

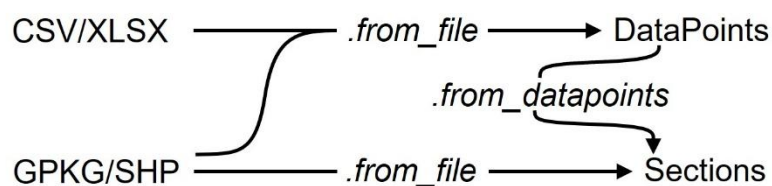


Fig 2. A graphical representation of Stage 1.

DataPoints

A DataPoints object is used for storing datapoints, where each datapoint is simply a point (in space and time) with associated data. For example, each datapoint could be a sighting with information on the number of individuals, a recording of environmental data such as Beaufort sea state and wind speed, or even a simple GPS point with nothing more than latitude and longitude.

The datapoints can be either continuous or sporadic. Continuous datapoints are recorded at frequent, regular intervals and so can be used to construct a trackline. For example, points recorded by a GPS every 10 seconds can be considered continuous. Sporadic datapoints are recorded at infrequent, irregular intervals and so cannot be used to construct a trackline. For example, sightings should typically be considered sporadic, as should environmental datapoints recorded only at the beginning and end of transects. The difference between continuous and sporadic datapoints does not affect how a given DataPoints object is made, but it does affect what can be done with that DataPoints object and so should be kept in mind.

Attributes

datapoints: a dataframe containing the datapoints

A geopandas.GeoDataFrame with the following key columns:

datapoint_id: a unique identifier for each datapoint

geometry: the geometry of each datapoint as a shapely.Point

datetime: the datetime of each point as a pd.Timestamp (or None if no datetimes are entered)

Additionally, the dataframe may contain any number of additional 'data columns' containing data of any type.

name: the name of the DataPoints object

The name of the DataPoints object as a string consisting of 'datapoints-' and the name of the imported file.

parameters: the parameters of the DataPoints object

Various parameters pertaining to the DataPoints object contained in a dictionary.

Class methods

from_file: make a DataPoints object from a GPKG, SHP, CSV, or XLSX file.

Takes a GPKG, SHP, CSV, or XLSX file that contains the datapoints and reformats it for subsequent processing by: renaming and reordering essential columns; if necessary, reprojecting it to a projected CRS; assigning each datapoint a unique ID.

If loading data from a CSV or XLSX, locations of datapoints must be stored in one of two ways:

two columns (x_col and y_col) containing x and y (e.g., longitude and latitude) coordinates

one column (geometry_col) containing points as WKT geometry objects

Parameters:

filepath : str

The path to the file containing the datapoints. Ensure that filepath includes the filename and the extension.

x_col : str, optional, default 'lon'

If importing a CSV or XLSX with x and y coordinates, the name of the column containing the x coordinate (e.g., longitude) of each datapoint.

y_col : str, optional, default 'lat'

If importing a CSV or XLSX with x and y coordinates, the name of the column containing the y coordinate (e.g., latitude) of each datapoint.

geometry_col : str, optional, default None

If importing a CSV or XLSX with points as WKT geometry objects, the name of the column containing the WKT geometry objects.

crs_import : str | int | pyproj.CRS, optional, default None

If importing a CSV or XLSX, the CRS of the coordinates/geometries. The CRS must be either: a pyproj.CRS; a string in a format accepted by pyproj.CRS.from_user_input (e.g., 'EPSG:4326'); or an integer in a format accepted by pyproj.CRS.from_user_input (e.g., 4326).

crs_working : str | int | pyproj.CRS, optional, default None

The CRS to be used for the subsequent processing. In most cases, must be a projected CRS that, preferably, preserves distance and uses metres. The CRS must be either: a pyproj.CRS; a string in a format accepted by pyproj.CRS.from_user_input (e.g., 'EPSG:4326'); or an integer in a format accepted by pyproj.CRS.from_user_input (e.g., 4326).

datetime_col : str, optional, default None

If applicable, the name of the column containing the datetime or date of each datapoint.

datetime_format : str, optional, default None

Optionally, the format of the datetimes as a string (e.g., "%Y-%m-%d %H:%M:%S").

It is possible to use format="ISO8601" if the datetimes meet ISO8601 (units in greatest to least order, e.g., YYYY-MM-DD) or format="mixed" if the datetimes have different formats (although not recommended as slow and risky).

tz_import : str | timezone | pytz.BaseTzInfo, optional, default None

If datetime_col is specified, the timezone of the datetimes contained within the column. The timezone must be either: a datetime.timezone; a string of a UTC code (e.g., 'UTC+02:00', 'UTC-09:30'); or a string of a timezone name accepted by pytz (e.g., 'Europe/Vilnius' or 'Pacific/Marquesas').

tz_working : str | timezone | pytz.BaseTzInfo, optional, default None

The timezone to be used for the subsequent processing. The timezone must be either: a datetime.timezone; a string of a UTC code (e.g., 'UTC+02:00', 'UTC-09:30'); or a string of a

timezone name accepted by pytz (e.g., 'Europe/Vilnius' or 'Pacific/Marquesas'). Note that `tz_import` must be specified if `tz_working` is specified.

`datapoint_id_col` : str, optional, default None

If applicable, the name of the column containing the datapoint IDs. The datapoint IDs must be unique.

`section_id_col` : str, optional, default None

If subsequently using `Sections.from_datapoints`, the name of the column containing the section IDs. Each individual section must have its own unique ID. All the datapoints that make up a given section must have the same value for section ID so that, when `Sections.from_datapoints` is run, they are grouped together to form a `LineString`. It is recommended that section IDs be codes consisting of letters and numbers and, optionally, underscores (e.g., 's001' or '20250710_s01').

Returns:

`DataPoints`

Returns a `DataPoints` object with three attributes: name, parameters, and datapoints.

open: open a saved DataPoints object

Open a `DataPoints` object that has previously been saved with `DataPoints.save()`.

Parameters:

`folder` : str

The path to the folder containing the saved files.

`basename` : str

The name of the `DataPoints` object that was saved (without the extension).

`crs_working` : str | int | `pyproj.CRS`, optional, default None

The CRS to be used for the subsequent processing. In most cases, must be a projected CRS that, preferably, preserves distance and uses metres. The CRS must be either: a `pyproj.CRS`; a string in a format accepted by `pyproj.CRS.from_user_input` (e.g., 'EPSG:4326'); or an integer in a format accepted by `pyproj.CRS.from_user_input` (e.g., 4326).

`tz_working` : str | timezone | `pytz.BaseTzInfo`, optional, default None

The timezone to be used for the subsequent processing. The timezone must be either: a `datetime.timezone`; a string of a UTC code (e.g., 'UTC+02:00', 'UTC-09:30'); or a string of a timezone name accepted by pytz (e.g., 'Europe/Vilnius' or 'Pacific/Marquesas').

Methods

plot: plot the datapoints

Makes a basic matplotlib plot of the datapoints in greyscale.

Parameters:

sections : Sections, optional, default None

Optionally, a Sections object with sections to be plotted with the datapoints.

save: save the datapoints

Saves the datapoints GeoDataFrame as a GPKG. The name of the saved file will be the name of the DataPoints object. Additionally, the parameters will be exported as a CSV with the same name plus '-parameters'.

Parameters:

folder : str

The path to the export folder where the exported files will be saved

crs_export : str | int | pyproj.CRS, optional, default None

The CRS to reproject the datapoints to before saving (only reprojects the datapoints that are saved and not the DataPoints object).

tz_export : str | timezone | pytz.BaseTzInfo, optional, default None

The timezone to convert the datapoints to before saving (only converts the datapoints that are saved and not the DataPoints object).

params : bool, default True

If True (the default), the parameters will be saved as a CSV, otherwise if False, the parameters will not be saved.

Sections

A Sections object is used for storing sections, where each section is a stretch of continuous, uniform survey track.

Defining sections

It is up to the user to define what constitutes a section and to designate the sections before processing with sampley (see below for more info on how to designate sections).

The key feature of sections is that they are continuous (i.e., without gaps). For example, a survey may initially result in a single long track, however, portions of this track are then removed as they do not meet certain requirements (e.g., weather conditions are poor or observers are off-watch). Now, what remains is a series of pieces of track with gaps in between. This series of pieces and gaps is not continuous and so cannot constitute a section, however, each individual piece is continuous and so can be a section.

It is also often desirable that sections be uniform, although what constitutes 'uniform' will vary between studies. Broadly speaking, uniformity is often considered in terms of parameters such as speed, observation effort, and weather conditions. It is up to the user to decide which parameters they deem relevant and to use them to define sections before importing data to sampley.

Designating sections (and section IDs)

If the sections are made from a GPKG or a SHP (with `Sections.from_file`), it is recommended that each section be represented by its own feature (e.g., a single linestring). Nevertheless, having multiple sections within a single feature (e.g., in a multi-linestring) is also allowed. Each individual section can have its own pre-defined unique section ID contained within the GPKG/SHP or, if not, section IDs will be automatically generated during importing.

If the sections are made via a DataPoints object (with `Sections.from_datapoints`), it is necessary to have a section ID column in the imported file specifying which section each datapoint belongs to. Each individual section must have its own unique ID. All the datapoints that make up a given section must have the same value for section ID so that, when `Sections.from_datapoints` is run, they are grouped together to form a LineString.

In either case (using `Sections.from_file` or `Sections.from_datapoints`), it is recommended that section IDs be codes consisting of letters and numbers and, optionally, underscores (e.g., s001 or 20250710_s01).

Attributes

sections: a dataframe containing the sections

A `geopandas.GeoDataFrame` with the following key columns:

`section_id`: a unique identifier for each section

`geometry`: the geometry of each section as a `shapely.LineString`

`datetime`: the datetime of the section as a `pd.Timestamp`

name: the name of the Sections object

The name of the Sections object as a string consisting of 'sections-' and the name of the imported file.

parameters: the parameters of the Sections object

Various parameters pertaining to the Sections object contained in a dictionary.

Class methods

from_file: make a Sections object from a GPKG or SHP file.

Takes a GPKG or SHP file that contains the sections as shapely.LineStrings and reformats it for subsequent processing by: renaming and reordering essential columns; if necessary, reprojecting it to a projected CRS; assigning each section a unique ID.

Sections can be made from CSV or XLSX files containing series of points by first making a DataPoints object and then using Sections.from_datapoints to make a Sections object.

Parameters:

filepath : str

The path to the file containing the sections. Ensure that filepath includes the filename and the extension.

crs_working : str | int | pyproj.CRS, optional, default None

The CRS to be used for the subsequent processing. In most cases, must be a projected CRS that, preferably, preserves distance and uses metres. The CRS must be either: a pyproj.CRS; a string in a format accepted by pyproj.CRS.from_user_input (e.g., 'EPSG:4326'); or an integer in a format accepted by pyproj.CRS.from_user_input (e.g., 4326).

datetime_col : str, optional, default None

The name of the column containing the datetime of each section.

datetime_format : str, optional, default None

Optionally, the format of the datetimes as a string (e.g., "%Y-%m-%d %H:%M:%S").

It is possible to use format="ISO8601" if the datetimes meet ISO8601 (units in greatest to least order, e.g., YYYY-MM-DD) or format="mixed" if the datetimes have different formats (although not recommended as slow and risky).

tz_import : str | timezone | pytz.BaseTzInfo, optional, default None

If datetime_col is specified, the timezone of the datetimes contained within the column. The timezone must be either: a datetime.timezone; a string of a UTC code (e.g., 'UTC+02:00', 'UTC-09:30'); or a string of a timezone name accepted by pytz (e.g., 'Europe/Vilnius' or 'Pacific/Marquesas').

tz_working : str | timezone | pytz.BaseTzInfo, optional, default None

The timezone to be used for the subsequent processing. The timezone must be either: a datetime.timezone; a string of a UTC code (e.g., 'UTC+02:00', 'UTC-09:30'); or a string of a

timezone name accepted by pytz (e.g., 'Europe/Vilnius' or 'Pacific/Marquesas'). Note that `tz_import` must be specified if `tz_working` is specified.

`section_id_col` : str, optional, default None

Optionally, the name of the column containing the section IDs. Each individual section must have its own unique ID. It is recommended that section IDs be codes consisting of letters and numbers and, optionally, underscores (e.g., 's001' or 20250710_s01').

Returns:

Sections

Returns a Sections object with three attributes: name, parameters, and sections.

from_datapoints: make a Sections object from a DataPoints object.

Takes a DataPoints object that contains sections as continuous series of Points and reformats it for subsequent processing by: converting each series of Points to a LineString; renaming and reordering essential columns. The CRS and timezone will be that of the DataPoints object.

Note that, when making the DataPoints object, it is necessary to specify `section_id_col`. Please see the documentation for Sections or for the `section_id_col` parameter under `DataPoints.from_file` for more information on section IDs and how they should be formatted.

Note that `Sections.from_datapoints` should only be used with continuous datapoints and not with sporadic datapoints (please see under DataPoints for details on continuous and sporadic datapoints).

Parameters:

`datapoints` : DataPoints

The DataPoints object that contains sections as series of points.

`cols` : dict | None, optional, default None

A dictionary whose keys are the names of any columns to keep and whose values are corresponding functions specifying what to do with those columns. For example, if each section has a pre-set period in a column called 'season', `cols` could be specified as {'season': 'first'} to keep the first value of season for each section.

`sortby` : str | list, optional, default None

When converting each series of Points to a LineString, the Points are joined one to the next (like a dot-to-dot). If the Points are not in the right order, the resulting LineString will be incorrect. If `sortby` is not specified, the Points will be joined in the order that they are in. To change this order, specify `sortby` as the name of a column or columns (e.g., 'datetime') to sort the datapoints by before the Points are converted to LineStrings.

Returns:

Sections

Returns a Sections object with three attributes: name, parameters, and sections.

open: open a saved Sections object

Open a Sections object that has previously been saved with `Sections.save()`.

Parameters:

`folder` : str

The path to the folder containing the saved files.

`basename` : str

The name of the Sections object that was saved (without the extension).

`crs_working` : str | int | pyproj.CRS, optional, default None

The CRS to be used for the subsequent processing. In most cases, must be a projected CRS that, preferably, preserves distance and uses metres. The CRS must be either: a pyproj.CRS; a string in a format accepted by pyproj.CRS.from_user_input (e.g., 'EPSG:4326'); or an integer in a format accepted by pyproj.CRS.from_user_input (e.g., 4326).

`tz_working` : str | timezone | pytz.BaseTzInfo, optional, default None

The timezone to be used for the subsequent processing. The timezone must be either: a datetime.timezone; a string of a UTC code (e.g., 'UTC+02:00', 'UTC-09:30'); or a string of a timezone name accepted by pytz (e.g., 'Europe/Vilnius' or 'Pacific/Marquesas').

Methods

plot: plot the sections

Makes a basic matplotlib plot of the sections in greyscale.

Parameters:

`datapoints` : DataPoints, optional, default None

Optionally, a DataPoints object with datapoints to be plotted with the sections.

save: save the sections

Saves the sections GeoDataFrame as a GPKG. The name of the saved file will be the name of the Sections object. Additionally, the parameters will be exported as a CSV with the same name plus '-parameters'.

Parameters:

`folder` : str

The path to the export folder where the exported files will be saved

`crs_export` : str | int | pyproj.CRS, optional, default None

The CRS to reproject the sections to before saving (only reprojects the sections that are saved and not the Sections object).

`tz_export` : str | timezone | pytz.BaseTzInfo, optional, default None

The timezone to convert the sections to before saving (only converts the sections that are saved and not the Sections object).

params : bool, default True

If True (the default), the parameters will be saved as a CSV, otherwise if False, the parameters will not be saved.

Stage 2 – delimiting

Stage 2 involves delimiting one or more of the following: periods, cells, segments, presences, presences zones, and absences. The approach(es) used and the data imported will determine which of these are necessary to delimit. The result of this stage is one or more objects that are referred to collectively as ‘delimiters’, i.e., objects that delimit samples (e.g., grid cells set the spatial limits of samples). There is no limit to the number of delimiters that can be made, thus enabling the user to implement different procedures and settings to determine the best option for data processing.

Paths

There are three sets of paths corresponding to the three approaches: grid, segment, and point.

Grid approach: A Cells object can be made from a DataPoints or a Sections object by using the Cells.delimit class method (Fig 3a). Similarly, a Periods object can be made from a DataPoints or a Sections object by using the Periods.delimit class method (Fig 3a).

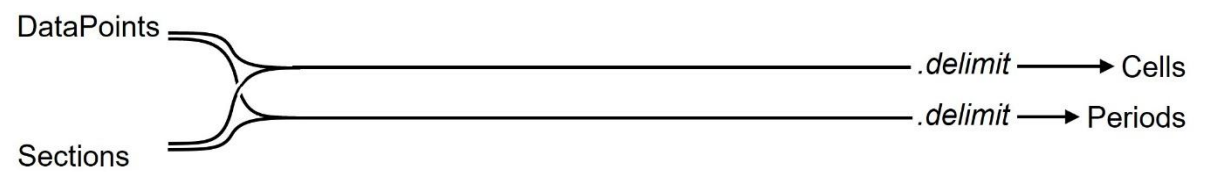


Fig 3a. A graphical representation of Stage 2 – grid approach.

Segment approach: A Segments object can be made from a Sections object by using the Segments.delimit class method (Fig 3b).

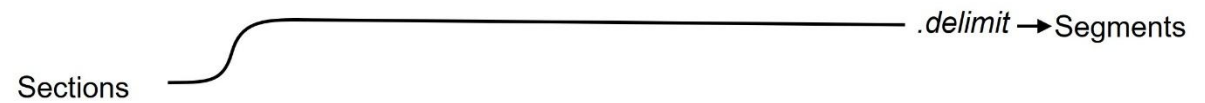
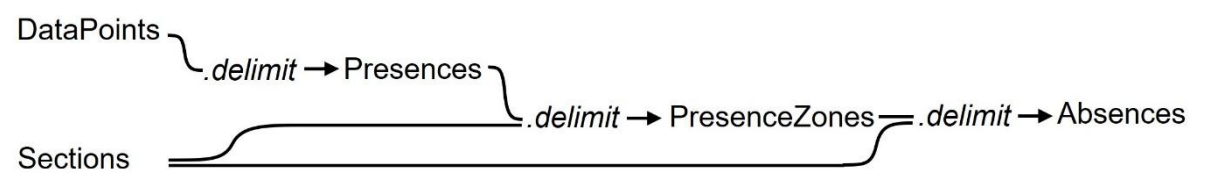


Fig 3b. A graphical representation of Stage 2 – segment approach.

Point approach: A Presences object can be made from a DataPoints object by using the Presences.delimit class method (Fig 3c). A PresenceZones object can be made from a Presences object and a Sections object by using the PresenceZones.delimit class method (Fig 3c). An Absences



object can then be made from a Sections object and a PresenceZones object with the Absences.delimit class method (Fig 3c).

Fig 3c. A graphical representation of Stage 2 – point approach.

Periods

Attributes

periods: a dataframe containing the periods

A pandas.DataFrame with the following key columns:

period_id: a unique identifier for each period consisting of 'p', the begin date, a dash, the number of units, and the unit (e.g., 'p2025-01-22-8d' is an 8 day period beginning the 22nd Jan 2025).

date_beg: the date of the beginning of each period as a pd.Timestamp

date_mid: the date of the midpoint of each period as a pd.Timestamp

date_end: the date of the end of each period as a pd.Timestamp

name: the name of the Periods object

The name of the Periods object as a string consisting of 'periods-', the number of units, and the units (e.g., 'periods-8d', meaning periods of 8 days).

parameters: the parameters of the Periods object

Various parameters pertaining to the Periods object contained in a dictionary.

Class methods

delimit: delimit temporal periods of a set number of units

From a given extent, number of units, and type of units, delimit temporal periods of regular length, e.g., 8 days, 2 months, or 1 year.

Temporal periods of irregular length (e.g., seasons) should be predefined and contained within a column of the imported data.

Parameters:

extent : Sections | DataPoints | pandas.DataFrame | tuple[list, str]

An object detailing the temporal extent over which the periods will be limited. Must be one of:

a Sections object whose sections GeoDataFrame has a 'datetime' column

a DataPoints object whose datapoints GeoDataFrame has a 'datetime' column

a pandas.DataFrame that has a 'datetime' column

a tuple containing two elements: a list of two datetimes and a timezone as a string (or None if no timezone is to be used)

num : int | float

The number of temporal units.

unit : str

The temporal units assigned with one of the following strings:

'day': days ('d' also accepted)

'month': months ('m' also accepted)

'year': years ('y' also accepted)

Returns:

Periods

Returns a Periods object with three attributes: name, parameters, and periods.

open: open a saved Periods object

Open a Periods object that has previously been saved with Periods.save().

Parameters:

folder : str

The path to the folder containing the saved files.

basename : str

The name of the Periods object that was saved (without the extension).

Methods

save: save the periods

Saves the periods DataFrame as a CSV. The name of the saved file will be the name of the Periods object. Additionally, the parameters will be exported as a CSV with the same name plus '-parameters'.

Parameters:

folder : str

The path to the export folder where the exported files will be saved

params : bool, default True

If True (the default), the parameters will be saved as a CSV, otherwise if False, the parameters will not be saved.

Cells

Attributes

cells: a dataframe containing the cells

A geopandas.GeoDataFrame with the following key columns:

cell_id: a unique identifier for each cell consisting of 'c', a unique number, a dash, 'r'/'h' for rectangular/hexagonal, the side length, and the units (e.g., 'c123-r5000m' is cell number 123 that is rectangular with 5000 metre sides).

polygon: the geometry of each cell as a shapely.Polygon

centroid: the centroid of each cell as a shapely.Point

name: the name of the Cells object

The name of the Cells object as a string consisting of 'cells-', 'r' / 'h' for rectangular / hexagonal and the side length (e.g., 'cells-r5000m', meaning cells that are rectangular with 5000 metre sides).

parameters: the parameters of the Cells object

Various parameters pertaining to the Cells object contained in a dictionary.

Class methods

delimit: delimit grid cells

From a given extent, variation, and side length, delimit rectangular or hexagonal grid cells of a regular size.

Parameters:

extent : Sections | DataPoints | geopandas.GeoDataFrame | tuple[list, str]

An object detailing the spatial extent over which the periods will be limited. Must be one of:

a Sections object

a DataPoints object

a geopandas.GeoDataFrame

a tuple containing two elements: a list containing the x min, y min, x max, and y max and a CRS

var : {'rectangular', 'hexagonal'}

The variation used to generate the cells. Must be one of the following:

'rectangular': make rectangular (square) cells ('r' also accepted)

'hexagonal': make hexagonal cells ('h' also accepted)

side : int | float

The side length of the rectangles/hexagons in the units of the CRS.

`coarse` : int, default None

If specified, cells will be made via a two-step process where the first step involves making coarse cells within which the cells are then made. This can speed up cell making for larger numbers of cells (e.g., >10 000), particularly if `select` is used.

The value of `coarse` is a factor that scales the size of the coarse cells relative to the cells (e.g., if `coarse` = 10, each coarse cell will be 10 times larger than each cell).

If using `coarse`, it is recommended to use `select` as well.

Note that `coarse` is only applicable if the extent is a `GeoDataFrame` and the variation is rectangular.

`buffer` : int | float, optional, default 0

The width of a buffer to be created around the extent to enlarge it and ensure that all the surveyed area is covered by the cells.

`select` : str, default None

Cells are initially made in a regular grid where all rows are the same length and all columns are the same length. If `select` is None (default), this is how they will be left. Alternatively, the method to use to select which cells to keep can be specified. Must be one of the following:

'intersects': only cells that intersect the extent are kept

'within': only cells that are within the extent are kept

'centroid': only cells whose centroid intersects the extent are kept

Note that `select` is only applicable if the extent is a `Sections` object, a `DataPoints` object, or a `GeoDataFrame`.

Returns:

Cells

Returns a `Cells` object with three attributes: `name`, `parameters`, and `cells`.

open: open a saved `Cells` object

Open a `Cells` object that has previously been saved with `Cells.save()`.

Parameters:

`folder` : str

The path to the folder containing the saved files.

`basename` : str

The name of the `Cells` object that was saved (without the extension).

`crs_working` : str | int | `pyproj.CRS`, optional, default None

The CRS to be used for the subsequent processing. In most cases, must be a projected CRS that, preferably, preserves distance and uses metres. The CRS must be either: a `pyproj.CRS`; a string in a format accepted by `pyproj.CRS.from_user_input` (e.g., 'EPSG:4326'); or an integer in a format accepted by `pyproj.CRS.from_user_input` (e.g., 4326).

Methods

plot: plot the cells

Makes a basic matplotlib plot of the cells.

Parameters:

`datapoints` : DataPoints, optional, default None

Optionally, a DataPoints object with datapoints to be plotted with the cells.

`sections` : Sections, optional, default None

Optionally, a Sections object with sections to be plotted with the cells.

save: save the cells

Saves the cells GeoDataFrame as two GPKGs: one of the polygons and one of the centroids. The names of the saved files will be the name of the Cells object plus '-polygons' and '-centroids', respectively. Additionally, the parameters will be exported as a CSV with the same name plus '-parameters'.

Parameters:

`folder` : str

The path to the export folder where the exported files will be saved

`crs_export` : str | int | pyproj.CRS, optional, default None

The CRS to reproject the cells to before saving (only reprojects the cells that are saved and not the Cells object). The CRS must be either: a pyproj.CRS; a string in a format accepted by pyproj.CRS.from_user_input (e.g., 'EPSG:4326'); or an integer in a format accepted by pyproj.CRS.from_user_input (e.g., 4326).

`params` : bool, default True

If True (the default), the parameters will be saved as a CSV, otherwise if False, the parameters will not be saved.

Segments

Attributes

segments: a dataframe containing the segments

A geopandas.GeoDataFrame with the following key columns:

segment_id: a unique identifier for each segments

segment_id: the ID of each segment consisting of 's', a unique number, a dash, 's'/'j'/'r' for simple/joining/redistribution variation, the target length, and the units (e.g., 's123-r1000m' is segment 123 made with the redistribution method and a target length of 1000 metres).

line: the geometry of the segment as a shapely.LineString

midpoint: the midpoint of the segment as a shapely.Point

section_id: the ID of the section from which the segment was cut

dfbsec_beg: the distance along the section where the segment begins

dfbsec_end: the distance along the section where the segment ends

name: the name of the Segments object

The name of the Segments object as a string consisting of 'segments-', 's' / 'j' / 'r' for simple / joining / redistribution variation, the target length, and the units (e.g., 'segments-r1000m', meaning segments made with the redistribution method and a target length of 1000 metres).

parameters: the parameters of the Segments object

Various parameters pertaining to the Segments object contained in a dictionary.

Class methods

delimit: delimit segments

With a given variation and target length, cut sections into segments.

Segments can be made with any one of three variations: the simple, joining, and redistribution variations. For all three variations, a target length is set. The variations differ in how they deal with the remainder – the length inevitably left over after dividing a section by the target length. Additionally, for the simple and joining variations, the location of the remainder / joined segment can be randomised (rather than always being at the end).

Parameters:

sections : Sections

The Sections object containing the sections from which the segments will be cut.

var : {'simple', 'joining', 'redistribution'}

The variation to use to make the segments. Must be one of the following:

'simple': the remainder is left as an independent segment ('s' also accepted)

'joining': the remainder, if under half the target length, is joined to another segment, otherwise it is left as an independent segment ('j' also accepted)

'redistribution': the length of the remainder is redistributed among all segments ('r' also accepted)

target : int | float

The target length of the segments in the units of the CRS.

rand : bool, optional, default False

If using the simple or joining variations, whether to randomise the location of the remainder / joined segment or not.

Returns:

Segments

Returns a Segments object with three attributes: name, parameters, and segments.

open: open a saved Segments object

Open a Segments object that has previously been saved with Segments.save().

Parameters:

folder : str

The path to the folder containing the saved files.

basename : str

The name of the Segments object that was saved (without the extension).

crs_working : str | int | pyproj.CRS, optional, default None

The CRS to be used for the subsequent processing. In most cases, must be a projected CRS that, preferably, preserves distance and uses metres. The CRS must be either: a pyproj.CRS; a string in a format accepted by pyproj.CRS.from_user_input (e.g., 'EPSG:4326'); or an integer in a format accepted by pyproj.CRS.from_user_input (e.g., 4326).

Methods

datetimes: get datetimes for the beginning, middle, and end of each segment

Get a datetime value for the beginning, middle, and end of each segment. This is only applicable to segments that were made from sections that were made from continuous datapoints with Sections.from_datapoints. Additionally, it requires that those datapoints have datetime values.

In the (likely) case that a segment begins/ends at some point between two datapoints, the begin/end time for that segment will be interpolated based on the distance from those two datapoints to the point at which the segment begins/ends assuming a constant speed.

Parameters:

datapoints : DataPoints

The DataPoints object, containing datetimes, that was used to make the Sections object that was used to make the Segments object.

plot: plot the segments

Makes a basic matplotlib plot of the segments.

Parameters:

`datapoints` : DataPoints, optional, default None

Optionally, a DataPoints object with datapoints to be plotted with the segments.

`sections` : Sections, optional, default None

Optionally, a Sections object with sections to be plotted with the segments.

save: save the segments

Saves the segments GeoDataFrame as two GPKGs: one of the lines and one of the midpoints. The names of the saved files will be the name of the Segments object plus '-lines' and '-midpoints', respectively. Additionally, the parameters will be exported as a CSV with the same name plus '-parameters'.

Parameters:

`folder` : str

The path to the export folder where the exported files will be saved

`crs_export` : str | int | pyproj.CRS, optional, default None

The CRS to reproject the segments to before saving (only reprojects the segments that are saved and not the Segments object). The CRS must be either: a pyproj.CRS; a string in a format accepted by `pyproj.CRS.from_user_input` (e.g., 'EPSG:4326'); or an integer in a format accepted by `pyproj.CRS.from_user_input` (e.g., 4326).

`params` : bool, default True

If True (the default), the parameters will be saved as a CSV, otherwise if False, the parameters will not be saved.

Presences

Attributes

presences: a dataframe containing the presences

A geopandas.GeoDataFrame containing the presences with the following key columns:

point_id: a unique identifier for each presence

point: the presence as a shapely.Point

date: the date of the presence as a pd.Timestamp

name: the name of the Presences object

The name of the Presences object as a string consisting of 'presences-' and the name of the DataPoints object from which the Presences object is derived.

parameters: the parameters of the Presences object

Various parameters pertaining to the Presences object contained in a dictionary.

Class methods

delimit: delimit presences

From a DataPoints object, make a Presences object.

There are two options for the datapoints: all rows are presences, in which case there is no need to specify presence_col, or only some rows are presences, in which case presence_col must be specified.

Parameters:

datapoints : DataPoints

The DataPoints object that contains the presences.

presence_col : str, optional, default None

The name of the column containing the values that determine which points are presences (e.g., a column containing a count of individuals). This column must contain only integers or floats.

Only needs to be specified if the DataPoints object includes points that are not presences.

block : str, optional, default None

Optionally, the name of a column that contains unique values to be used to separate the presences into blocks. These blocks can then be used later when generating absences.

Returns:

Presences

Returns a Presences object with three attributes: name, parameters and presences.

open: open a saved Presences object

Open a Presences object that has previously been saved with Presences.save().

Parameters:**folder** : str

The path to the folder containing the saved files.

basename : str

The name of the Presences object that was saved (without the extension).

crs_working : str | int | pyproj.CRS, optional, default None

The CRS to be used for the subsequent processing. In most cases, must be a projected CRS that, preferably, preserves distance and uses metres. The CRS must be either: a pyproj.CRS; a string in a format accepted by pyproj.CRS.from_user_input (e.g., 'EPSG:4326'); or an integer in a format accepted by pyproj.CRS.from_user_input (e.g., 4326).

Methods*plot: plot the presences*

Makes a basic matplotlib plot of the presences.

save: save the presences

Saves the presences as a GPKG file. The name of the saved file will be the name of the Presences object plus '-point'. Additionally, the parameters will be exported as a CSV with the same name plus '-parameters'.

Parameters:**folder** : str

The path to the export folder where the exported files will be saved

crs_export : str | int | pyproj.CRS, optional, default None

The CRS to reproject the presences to before saving (only reprojects the presences that are saved and not the Presences object). The CRS must be either: a pyproj.CRS; a string in a format accepted by pyproj.CRS.from_user_input (e.g., 'EPSG:4326'); or an integer in a format accepted by pyproj.CRS.from_user_input (e.g., 4326).

params : bool, default True

If True (the default), the parameters will be saved as a CSV, otherwise if False, the parameters will not be saved.

PresenceZones

Attributes

presencezones: a dataframe containing the presences zones

A geopandas.GeoDataFrame with the following key columns:

section_id: a unique identifier for the section to which the presence zones correspond (in the case that all sections have identical presence zones, this column will contain a single value: 'all')

presencezones: the geometry of each set of presences zones as a shapely.MultiPolygon

name: the name of the PresenceZones object

The name of the PresenceZones object as a string consisting of 'presencezones-', the spatial threshold, its units, and, if used, the temporal threshold and its units.

parameters: the parameters of the PresenceZones object

Various parameters pertaining to the PresenceZones object contained in a dictionary.

Class methods

delimit: delimit presences zones

From the presences, use a spatial and, optionally, temporal threshold to make presences zones.

Presence zones are zones around presences that are deemed to be 'occupied' by the animals. Absences will not be generated within the presence zones, thus they serve to ensure that absences are generated sufficiently far from presences.

Spatial and temporal thresholds determine the extent of the presence zones. The spatial threshold represents the radius and the temporal threshold the number of units (e.g., days) before and after that of the presence. For example, a spatial threshold of 10 000 m and a temporal threshold of 5 days means that no absence will be generated within 10 000 m and 5 days of any presence while a spatial threshold of 10 000 m and a temporal threshold of 0 days means that no absences will be generated within 10 000 m of and on the same day as any presence. Note that not entering any temporal threshold completely removes the temporal dimension, and so differs from entering a temporal threshold of 0.

Note that the presence zones correspond to sections – specifically, the sections that they overlap spatially and, optionally, temporally with, as determined by the spatial and temporal thresholds.

Parameters:

presences : Presences

The Presences object containing the presences from which the presences zones are to be made.

sections : Sections

The Sections object containing the sections to which the presences zones correspond.

`sp_threshold` : int | float, optional, default None

The spatial threshold to use for making the presences zones in the units of the CRS.

`tm_threshold` : int | float, optional, default None

The temporal threshold to use for making the presences zones in the units set with `tm_unit`.

`tm_unit` : str, optional, default 'day'

The temporal units to use for making the presences zones. One of the following:

'year': year (all datetimes from the same year will be given the same value)

'month': month (all datetimes from the same month and year will be given the same value)

'day': day (all datetimes with the same date will be given the same value)

'hour': hour (all datetimes in the same hour on the same date will be given the same value)

'moy': month of the year (i.e., January is 1, December is 12 regardless of the year)

'doy': day of the year (i.e., January 1st is 1, December 31st is 365 regardless of the year)

Returns:

PresenceZones

Returns a PresenceZones object with three attributes: name, parameters, and presencezones.

open: open a saved PresenceZones object

Open an PresenceZones object that has previously been saved with `PresenceZones.save()`.

Parameters:

`folder` : str

The path to the folder containing the saved files.

`basename` : str

The name of the PresenceZones object that was saved (without the extension).

`crs_working` : str | int | pyproj.CRS, optional, default None

The CRS to be used for the subsequent processing. In most cases, must be a projected CRS that, preferably, preserves distance and uses metres. The CRS must be either: a `pyproj.CRS`; a string in a format accepted by `pyproj.CRS.from_user_input` (e.g., 'EPSG:4326'); or an integer in a format accepted by `pyproj.CRS.from_user_input` (e.g., 4326).

Methods

plot: plot the presences zones

Makes a basic matplotlib plot of the presences zones.

Parameters:

sections : Sections, optional, default None

Optionally, a Sections object with sections to be plotted with the presences zones.

presences : Presences, optional, default None

Optionally, a Presences object with presences to be plotted with the presences zones.

save: save the presences zones

Saves the presences zones GeoDataFrame as a GPKG. The name of the saved file will be the name of the PresenceZones object. Additionally, the parameters will be exported as a CSV with the same name plus '-parameters'.

Parameters:

folder : str

The path to the export folder where the exported files will be saved

crs_export : str | int | pyproj.CRS, optional, default None

The CRS to reproject the presences zones to before saving (only reprojects the presences zones that are saved and not the PresenceZones object). The CRS must be either: a pyproj.CRS; a string in a format accepted by pyproj.CRS.from_user_input (e.g., 'EPSG:4326'); or an integer in a format accepted by pyproj.CRS.from_user_input (e.g., 4326).

params : bool, default True

If True (the default), the parameters will be saved as a CSV, otherwise if False, the parameters will not be saved.

Absences

Attributes

absences: a dataframe containing the absences

A geopandas.GeoDataFrame containing the absences with the following key columns:

point_id: a unique identifier for each absence

point: the absence as a shapely.Point

date: the date of the absence as a pd.Timestamp

section_id: the ID of the section along which or from which the absence was generated

dfbs: the distance from the beginning of the sections to the absence (used as a coordinate to locate the absence)

Additionally, if the 'from-the-line' variation is used:

point_al: a point corresponding to each point in the 'point' column but along the survey track

name: the name of the Absences object

The name of the Absences object as a string consisting of 'absences-', 'a' / 'f' for along-the-line / from-the-line variation, and the name of the PresenceZones object.

parameters: the parameters of the Absences object

Various parameters pertaining to the Absences object contained in a dictionary.

Class methods

delimit: delimit the absences

Absences can be generated by one of two variations: the 'along-the-line' variation or the 'from-the-line' variation.

In the along-the-line variation, each absence is generated by randomly placing a point along the survey track, provided it is not within the corresponding presences zones.

In the from-the-line variation, each absence is generated by randomly placing a first (temporary) point along the survey track and then placing a second point a certain distance from the first point perpendicular to the track, provided that this second point is not within the corresponding presences zones. This second point is the absence. The distance from the track is selected from a list of candidate distances that can be generated in any way, including from a predefined distribution (e.g., a detection function) by using the function generate_dfbs.

Absences are made with a 'hit-and-miss' approach: points are made irrespective of presences zones and then kept if they are not within the presence zones (hit) and discarded if they are within the presence zones (miss). This means that to generate a certain number of absences, it is typically necessary to generate a larger number of points to account for the misses. The maximum number of points that can be generated is calculated by multiplying the target by the parameter limit (e.g., if target=100 and

limit=10, a maximum of 1000 points can be generated). If the maximum number is reached (or the target number of absences is reached), further absence generation is abandoned and those absences that have been made are returned. This is a safety catch to prevent absence generation from entering an endless loop where all (or nearly all) points are misses.

Parameters:

sections : Sections

The Sections object containing the sections used to generate the absences.

presencezones : PresenceZones

The PresenceZones object containing the presences zones used to generate the absences.

var : {'along', 'from'}

The variation to use to generate the absences. Must be one of the following:

'along': along-the-line - the absences are generated by randomly placing a point along the surveyed lines ('a' also accepted)

'from': from-the-line - the absences are generated by, firstly, randomly placing a point along the line and then, secondly, placing a point a certain distance from the first point perpendicular to the line ('f' also accepted)

target : int | float

The total number of absences to be generated.

Note that if using block, the number of absences generated will likely be slightly higher than the target due to rounding.

Note that if using block and how='presences', the target is a factor to multiply the number of presences by.

Note that, during thinning (optionally conducted later on a Samples object), some absences may be removed so, to account for this, the target should be set higher than the final number desired.

limit : int, optional, default 10

The value that is multiplied by the target to get the maximum potential number of points (e.g., if target=100 and limit=10, a maximum of 1000 points can be generated). If the maximum number is reached (or the target number of absences is reached), further absence generation is abandoned and those absences that have been made are returned.

dfls : list[int | float], optional, default None

If using the from-the-line variation, a list of candidate distances from the line to use when generating absences. For each absence, one of these distances will be chosen at random and used to place the absence at that distance from the survey line. These distances can be generated in any way, including from a predefined distribution (e.g., a detection function) with the function generate_dfls.

block : str, optional, default None

Optionally, the name of a column in the sections that contains unique values to be used to separate the generation of absences into blocks. For example, to generate absences on a yearly basis or on a regional basis. If using block, how must also be specified.

how : str, optional, default None

If using block, how the number of absences to be generated per block is calculated. Must be one of the following:

'target' : the number of absences per block will be equal to the target

'average': the number of absences for all blocks will be the target divided by the number of blocks (rounded up if there is a remainder)

'effort': the number of absences will be the target divided proportionally by the amount of survey effort (measured as length of the sections) per block

'presences': the number of absences per block will be equal to the corresponding number of presences multiplied by the target (e.g., if a block has 19 presences and target=2, then 38 absences will be generated for that block); note that presences must also be entered if using this option

presences : Presences, optional, default None

If using block and how='presences', the Presences object on which to base the number of absences. Note that the presences must contain the same block column as the sections.

Returns:

Absences

Returns an Absences object with three attributes: name, parameters and absences.

open: open a saved Absences object

Open an Absences object that has previously been saved with Absences.save().

Parameters:

folder : str

The path to the folder containing the saved files.

basename : str

The name of the Absences object that was saved (without the extension).

crs_working : str | int | pyproj.CRS, optional, default None

The CRS to be used for the subsequent processing. In most cases, must be a projected CRS that, preferably, preserves distance and uses metres. The CRS must be either: a pyproj.CRS; a string in a format accepted by pyproj.CRS.from_user_input (e.g., 'EPSG:4326'); or an integer in a format accepted by pyproj.CRS.from_user_input (e.g., 4326).

Methods

plot: plot the absences

Makes a basic matplotlib plot of the absences.

Parameters:

presencezones : PresenceZones, optional, default None

Optionally, an PresenceZones object with presences zones to be plotted with the absences.

save: save the absences

Saves the absences as a GPKG file. The name of the saved file will be the name of the Absences object plus '-point'. Additionally, the parameters will be exported as a CSV with the same name plus '-parameters'. Parameters:

folder : str

The path to the export folder where the exported files will be saved

crs_export : str | int | pyproj.CRS, optional, default None

The CRS to reproject the absences to before saving (only reprojects the absences that are saved and not the Absences object). The CRS must be either: a pyproj.CRS; a string in a format accepted by pyproj.CRS.from_user_input (e.g., 'EPSG:4326'); or an integer in a format accepted by pyproj.CRS.from_user_input (e.g., 4326).

params : bool, default True

If True (the default), the parameters will be saved as a CSV, otherwise if False, the parameters will not be saved.

Stage 3 – sampling

The delimiters made in Stage 2 set the limits of the samples, however they do not contain any data. In this sense, they can be considered as empty containers waiting to be filled. Stage 3 concerns allocating data contained in one or more DataPoints and/or Sections objects to the delimiters (i.e., filling containers with their corresponding data).

Paths

Grid approach: A Samples object can be made from a DataPoints object, a Cells object, and, optionally, a Periods object by using the Samples.grid class method. Additionally, a Samples object with measures of survey effort can be made from a Sections object, a Cells object, and, optionally, a Periods object by using the Samples.grid_se class method. Finally, a Samples object can be made from multiple other Samples objects by using the Samples.merge class method (Fig 4a).

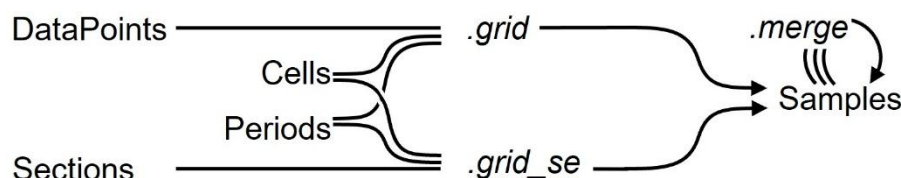


Fig 4a. A graphical representation of Stage 3 – grid approach.

Segment approach: A Samples object can be made from a DataPoints object and a Segments object by using the Samples.segment class method. Additionally, a Samples object with measures of survey effort can be made from a Segments object by using the Samples.segment_se class method. Finally, a Samples object can be made from multiple other Samples objects by using the Samples.merge class method (Fig 4b).

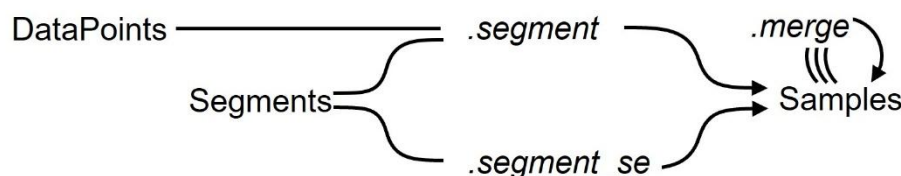


Fig 4b. A graphical representation of Stage 3 – segment approach.

Point approach: A Samples object can be made from a DataPoints object, a Presences object and an Absences object by using the Samples.point class method (Fig 4c).

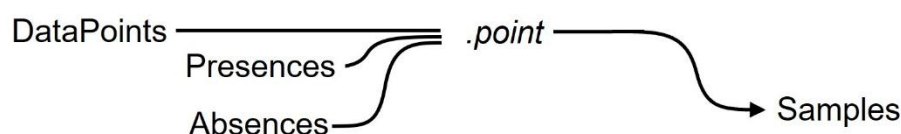


Fig 4c. A graphical representation of Stage 3 – point approach.

Samples

Attributes

samples: a dataframe containing the samples

A geopandas.GeoDataFrame with the key columns of the delimiters (i.e., Periods, Cells, Segments, Presences, or Absences) used to generate the samples as well as any number of data columns.

name: the name of the Samples object

The name of the Samples object as a string consisting of 'samples-' and an amalgamation of the names of the delimiters (i.e., Periods, Cells, Segments, Presences, or Absences) used to generate the samples.

parameters: the parameters of the Samples object

Various parameters pertaining to the Samples object contained in a dictionary.

assigned: the datapoints or sections with their allocated delimiter

Depending on the approach used, a geopandas.GeoDataFrame of the datapoints or sections with an additional column (or columns) indicating the Periods, Cells, or Segments that each datapoint or section has been allocated to. Mainly for the purposes of verifying the allocation process.

Class methods

grid: resample datapoints using the grid approach

Determines which cell and period each datapoint lies within and then groups together datapoints that lie within the same cell and period. As multiple datapoints may lie within the same cell and period, it is necessary to treat them in some way (e.g., average them, sum them). The parameter cols dictates how each column is to be treated.

Parameters:

datapoints : DataPoints

The DataPoints object.

cols : dict

A dictionary indicating how to treat each of the data columns. The dictionary should have the format:

```
{'COLUMN': FUNCTION,
 'COLUMN': FUNCTION}
```

...where COLUMN is the name of a given column as a string (e.g., 'bss') and FUNCTION is a function to apply to the values in that column when they are grouped together (e.g., 'mean').

Functions include those available for pandas.groupby plus some custom functions provided here. For example:

'mean' - get the mean
 'min' - get the minimum
 'max' - get the maximum
 mode - get the mode
 'sum' - sum the values
 'count' - count how many have a value (0s counted, NAs ignored)
 count_nz - count how many have a value (0s not counted, NAs ignored)
 pa - convert numeric values to binary presence-absence (0s not counted, NAs ignored)
 list - list the values

Note that some functions have quotation marks, while others do not. Note that some functions differ in how they treat NA values (missing values) and 0s. It is not necessary to specify all columns, but any columns not specified will not be retained. Each column can only be specified once.

cells : Cells

The Cells object.

periods : Periods | str | None, optional, default None

One of the following:

a Periods object
 a string indicating the name of the column in datapoints containing pre-set periods
 None

full : bool, optional, default False

If False, only those cell-period combinations that have at least one datapoint will be included in samples. If True, all possible cell-period combinations will be included in samples (note that this may result in a large number of samples that have no data).

Returns:

Samples

Returns a Samples object with four attributes: name, parameters, samples, and assigned.

Examples:

For a set of datapoints with a column of counts of individuals, 'individuals', and a column of values for Beaufort sea state (BSS), 'bss', the parameter cols could be set to the following in order to sum the individuals observed per sample and get the mean BSS per sample:

```
cols={'individuals': 'sum', 'bss': 'mean'}
```

segment: resample datapoints using the segment approach

Determines which segment each datapoint corresponds to and then groups together datapoints that correspond to the same segment. As multiple datapoints may correspond to the same segment, it is necessary to treat them in some way (e.g., average them, sum them). The parameter cols dictates how each column is to be treated.

Parameters:

datapoints : DataPoints

The DataPoints object.

segments : Segment

The Segments object.

cols : dict

A dictionary indicating how to treat each of the data columns. The dictionary should have the format:

```
{'COLUMN': FUNCTION,
 'COLUMN': FUNCTION}
```

...where COLUMN is the name of a given column as a string (e.g., 'bss') and FUNCTION is a function to apply to the values in that column when they are grouped together (e.g., 'mean').

Functions include those available for pandas.groupby plus some custom functions provided here. For example:

'mean' - get the mean

'min' - get the minimum

'max' - get the maximum

mode - get the mode

'sum' - sum the values

'count' - count how many have a value (0s counted, NAs ignored)

count_nz - count how many have a value (0s not counted, NAs ignored)

pa - convert numeric values to binary presence-absence (0s not counted, NAs ignored)

list - list the values

Note that some functions have quotation marks, while others do not. Note that some functions differ in how they treat NA values (missing values) and 0s. It is not necessary to specify all columns, but any columns not specified will not be retained. Each column can only be specified once.

how : { 'line', 'midpoint', 'datetime', 'dfb' }

An option specifying how to determine which segment each datapoint corresponds to. Must be one of the following:

line: each datapoint is matched to the nearest segment that has the same date

midpoint: each datapoint is matched to the segment with the nearest midpoint that has the same date

datetime: each datapoint is matched to a segment based on the datetime of the datapoint and the beginning datetimes of the segments (note that Segments.dates must be run before; note also that, if multiple surveys are run simultaneously, they will need to be processed separately to avoid datapoints from one survey being allocated to segments from another due to temporal overlap)

dfb: each datapoint is matched to a segment based on the distance it is located from the start of the sections lines (only applicable for matching segments that were made from sections that were made from datapoints with Sections.from_datapoints and those datapoints)

Returns:

Samples

Returns a Samples object with four attributes: name, parameters, samples, and assigned.

Examples:

For a set of datapoints that has a column of counts of individuals, 'individuals', and a column of values for Beaufort sea state (BSS), 'bss', the parameter cols could be set to the following in order to sum the individuals observed per sample and get the mean BSS per sample:

```
cols={'individuals': 'sum', 'bss': 'mean'}
```

point: resample datapoints using the point approach

Concatenates the presences and absences and assigns them presence-absence values of 1 and 0, respectively.

Additionally, and optionally, for each presence, gets data from its corresponding datapoint (i.e., the datapoint from which the presence was derived).

Additionally, and optionally, for each absence, gets the datapoint prior to it and assigns to the absence that datapoint's data. The ID of the prior datapoint is also added to the datapoint_id column. Note that this is only applicable if absences were generated from sections that were, in turn, made from datapoints with Sections.from_datapoints and those corresponding datapoints.

Parameters:

presences : Presences

The Presences object.

absences : Absences

The Absences object.

datapoints_p : DataPoints, optional, default None

If adding data to the presences, the DataPoints object containing the data. If specified, data will be added to the presences, if not specified, data will not be added to the presences.

cols_p : list, optional, default None

If adding data to the presences, a list indicating which data columns in datapoints_p to add to the presences.

datapoints_a : DataPoints, optional, default None

If adding data to the absences, the DataPoints object containing the data. If specified, data will be added to the absences, if not specified, data will not be added to the absences. Note that adding data to absences is only applicable for absences that were generated from sections that were made from datapoints and those datapoints.

cols_a : list, optional, default None

If adding data to the absences, a list indicating which data columns in datapoints_a to add to the absences.

block : str, optional, default None

If adding data to absences, optionally, the name of a column in the datapoints and absences that contains unique values to be used to separate the datapoints and absences into blocks in order to speed up the assigning of data to absences. If block was used to delimit absences, it must be used here, if adding data to absences.

Returns:

Samples

Returns a Samples object with three attributes: name, parameters, and samples.

grid_se: measure survey effort using the grid approach

Measures the amount of survey track that lies within each cell-period combination to get a measure of survey effort. Survey effort per cell-period can be measured in two ways:

length - length of the survey track in each cell-period

area - area of the buffered survey track in each cell-period

Moreover, each of these ways can be measured using Euclidean or geodesic measurements, as determined by the parameter euc_geo. Geodesic measurements will be more precise but take longer to run. Multiple measures of survey effort can be calculated simultaneously. If the parameter length is True, length will be measured. If the parameter esw is specified, area will be measured.

Parameters:

sections : Sections

The Sections object.

cells : Cells

The Cells object.

periods : Periods | str | None, optional, default, None

One of the following:

a Periods object

a string indicating the name of the column in datapoints containing pre-set periods

None

length : bool, optional, default True

If True, the length of survey track in each cell-period combination will be measured.

esw : int | float, optional, default None

Optionally, the one-sided effective stripwidth (ESW). If a value is given, the area of survey track in each cell-period combination will be measured. Note that ESW is one-sided.

euc_geo : {'euclidean', 'geodesic', 'both'}, optional, default 'euclidean'

The type of measurement. Must be one of the following: 'euclidean', 'geodesic', or 'both'.

full : bool, optional, default False

If False, only those cell-period combinations that have at least some survey effort will be included in samples. If True, all possible cell-period combinations will be included in samples (note that this may result in a large number of samples that have no data).

Returns

Samples

Returns a Samples object with four attributes: name, parameters, samples, and assigned. Within the samples attribute, the survey effort measures will be contained in the following columns (if applicable):

se_length: survey effort measured as length with Euclidean distances

se_area: survey effort measured as area with Euclidean distances

se_length_geo: survey effort measured as length with geodesic distances

se_area_geo: survey effort measured as area with geodesic distances

segment_se: measure survey effort using the segment approach

Measures the amount of survey effort per segment. Survey effort per segment can be measured in three ways:

length - length of the segment

area - length of the segment multiplied by a one-sided ESW multiplied by 2

effective area - length of the segment multiplied by a one-sided area under a detection function multiplied by 2

Moreover, each of these ways can be measured using Euclidean or geodesic measurements, as determined by the parameter euc_geo. Geodesic measurements will be more precise but take longer to run. Multiple measures of survey effort can be calculated simultaneously. If the parameter length is True, length will be measured. If the parameter esw is specified, area will be measured.

Parameters:

segments : Segments

The Segments object.

length : bool, optional, default True

If True, the length of each segment will be measured.

esw : int | float, optional, default None

Optionally, the one-sided effective stripwidth (ESW). If a value is given, the area of each segment will be measured. Note that ESW is one-sided.

audf : int | float, optional, default None

Optionally, the one-sided area under detection function (AUDF). If a value is given, the effective area of each segment will be measured. Note that AUDF is one-sided.

euc_geo : {'euclidean', 'geodesic', 'both'}, optional, default 'euclidean'

The type of measurement. Must be one of the following: 'euclidean', 'geodesic', or 'both'.

Returns

Samples

Returns a Samples object with three attributes: name, parameters, and samples. Within the samples attribute, the survey effort measures will be contained in the following columns (if applicable):

- se_length: survey effort measured as length with Euclidean distances
- se_area: survey effort measured as area with Euclidean distances
- se_effective: survey effort measured as effective area with Euclidean distances
- se_length_geo: survey effort measured as length with geodesic distances
- se_area_geo: survey effort measured as area with geodesic distances
- se_effective_geo: survey effort measured as effective area with geodesic distances

merge: merge multiple Samples objects together

Merge multiple Samples objects into a single new Samples object. Each Samples object should be entered as a parameter with a unique name of the user's choosing (note that this name will be used to name the merged Samples object). Only Samples objects made with the grid or segment approach can be merged (i.e., Samples objects must be generated by one or more of Samples.grid, Samples.segment, Samples.grid_se, or Samples.segment_se, but not Samples.point).

Parameters:

****kwargs :**

Any number of Samples objects each entered as a parameter with a unique name of the user's choosing.

Returns:

Samples

Returns a Samples object with three attributes: name, parameters, and samples.

Methods

thin: spatially, temporally, or spatiotemporally thin the samples

Spatially, temporally, or spatiotemporally thin the samples so that no two samples are within some spatial threshold and/or within some temporal threshold of each other.

If only a spatial threshold is specified, spatial thinning will be conducted. If only a temporal threshold is specified, temporal thinning will be conducted. If both a spatial and a temporal threshold are specified, spatiotemporal thinning will be conducted.

Modifies the attribute `Samples.samples` to contain only those samples kept after thinning. Those samples removed after thinning are placed in a new attribute: `Samples.removed`.

Parameters:

`coords` : str, optional, default None

The name of the column in the samples containing the geometries to use for thinning. Depending on the approach used to generate the samples, this column should be one of the following:

grid approach: 'centroid'

segment approach: 'midpoint'

point approach: 'point'

`sp_threshold` : int | float, optional, default None

The spatial threshold to use for spatial and spatiotemporal thinning in the units of the CRS.

`datetimes` : str, optional, default None

The name of the column in the samples containing the datetimes to use for thinning. This column should be 'datetimes'.

`tm_threshold` : int | float, optional, default None

The temporal threshold to use for temporal and spatiotemporal thinning in the units set with `tm_unit`.

`tm_unit` : str, optional, default 'day'

The temporal units to use for temporal and spatiotemporal thinning. One of the following:

'year': year (all datetimes from the same year will be given the same value)

'month': month (all datetimes from the same month and year will be given the same value)

'day': day (all datetimes with the same date will be given the same value)

'hour': hour (all datetimes in the same hour on the same date will be given the same value)

'moy': month of the year (i.e., January is 1, December is 12 regardless of the year)

'doy': day of the year (i.e., January 1st is 1, December 31st is 365 regardless of the year)

block : str, optional, default None

Optionally, the name of a column that contains unique values to be used to separate the samples into blocks that will be thinned independently.

reproject: reproject the samples

Reprojects the samples GeoDataFrame to a target CRS.

Parameters:

crs_target : str | int | pyproj.CRS, optional, default None

The CRS to reproject the samples to.

coords: extracts the coordinates from the centroids, midpoints, or points

Extracts the coordinates from the centroids, midpoints, or points and puts them in two new columns suffixed with '_lon' and '_lat' or '_x' and '_y'.

save: save the samples

Saves the samples GeoDataFrame as a GPKG, a CSV, or both. The name of the saved file(s) will be the name of the Samples object. Additionally, the parameters will be exported as a CSV with the same name plus '-parameters'.

Parameters:

folder : str

The path to the export folder where the exported files will be saved

filetype : {'gpkg', 'csv', 'both'}, optional, default 'gpkg'

The type of file that the sections will be saved as.

gpkg: GeoPackage

csv: CSV

both: GeoPackage and CSV

crs_export : str | int | pyproj.CRS, optional, default None

Optionally, the CRS to reproject the samples to before saving (only reprojects the samples that are saved and not the Samples object).

coords : bool, optional, default False

If True, x and y coordinates will be extracted from the centroid, midpoint, or point geometries and put in separate columns. This may facilitate subsequent extraction of data from external sources.

params : bool, default True

If True (the default), the parameters will be saved as a CSV, otherwise if False, the parameters will not be saved.

Additional functions

These are additional functions that may optionally be used as part of the Stages. They are not tied to any Class and so should be used independently. They must be imported with one of the following lines:

To import all functions: `from sampley.functions import *`

To import a specific function: `from sampley.functions import FUNCTION`
 where FUNCTION is the name of the desired function (e.g., `generate_dfls`)

pa: convert count (of individuals or sightings) to binary presence-absence value

To be used within the `cols` argument of `Samples.grid` and `Samples.segment` (or within the standard pandas groupby function). Takes a column where each value is a count (e.g., the number of individuals per sighting) and converts it to a binary presence-absence column (presence: 1; absence: 0). The column of counts can contain NA values.

count_nz: counts non-zero and non-NA values

To be used within the `cols` argument of `Samples.grid` and `Samples.segment` (or within the standard pandas groupby function). Takes a column and counts the number of values that are non-zero and non-NA.

generate_dfls: generate distances from a line

Generates distances from a line between 0 and the specified effective stripwidth (ESW) at intervals set by interval. If `dfunc` is specified, the distances will be based on probabilities set by `dfunc`, else probabilities will be even for all intervals.

Parameters:

`number: int | float`

The number of distances to generate.

`esw: int | float`

The effective stripwidth (i.e., the maximum distance from the line).

`interval: int | float`

The minimum interval between potential distances. For example, if `interval=0.1`, potential distances will be: 0, 0.1, 0.2, 0.3, etc...

`dfunc: typing.Callable, optional, default None`

Optionally, a callable function (e.g., a detection function), in which case, distances will be generated based on probabilities derived from the function. The function should be predefined then entered to `generate_dfls`. If not specified, distances will be evenly distributed between 0 and the ESW.

Returns:

`list`

The distances from the line as a list of integers or floats.

Example:

```
number = 100000
esw = 2000
interval = 1
def dfunc(x): return exp((-x**2) / (2*500**2))
dfls = generate_dfls(number=number, esw=esw, interval=interval, dfunc=dfunc)
```

calculate_area_udf: calculate area under detection function.

Calculates the area under a detection function between 0 and the specified effective stripwidth (ESW). If dfunc is specified, the area will be based on dfunc, else the area will be based on an even probability across intervals.

Parameters:

esw: int | float

The effective stripwidth (i.e., the maximum distance from the line).

interval: int | float

The minimum interval between potential distances. For example, if interval=0.1, potential distances will be: 0, 0.1, 0.2, 0.3, etc...

dfunc: typing.Callable, optional, default None

Optionally, a callable function (e.g., a detection function), in which case, distances will be generated based on probabilities derived from the function. The function should be predefined then entered to generate_dfls. If not specified, distances will be evenly distributed between 0 and the ESW.

Returns:

integer | float

The area under the detection function as an integer or float.

Example:

```
esw = 2000
interval = 1
def dfunc(x): return exp((-x**2) / (2*500**2))
area = calculate_area_udf(esw=esw, interval=interval, dfunc=dfunc)
```